



Submitted to:-

Engr. Waqas Saleem

Submitted by:-

Hafsa Khalid

Reg no:-

2023-BSE-021

Section:-

5 A

Cloud Computing

Assignment # 02

Advanced Terraform & Nginx Multi-Tier Architecture

```
● @Hafsa-Khalid021 → /workspaces/Assignment2 (main) $ mkdir Assignment2
● @Hafsa-Khalid021 → /workspaces/Assignment2 (main) $ cd Assignment2
● @Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) $ pwd
/workspaces/Assignment2/Assignment2
● @Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) $ mkdir modules scripts screenshots docs
mkdir modules/networking modules/security modules/webserver
mkdir screenshots/part1 screenshots/part2 screenshots/part3 screenshots/part4 screenshots/part5 screenshots/part6 screenshots/bonus
● @Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) $ touch main.tf variables.tf outputs.tf locals.tf terraform.tfvars
touch README.md .gitignore

touch modules/networking/main.tf modules/networking/variables.tf modules/networking/outputs.tf
touch modules/security/main.tf modules/security/variables.tf modules/security/outputs.tf
touch modules/webserver/main.tf modules/webserver/variables.tf modules/webserver/outputs.tf

touch scripts/nginx-setup.sh scripts/apache-setup.sh
```

@Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) \$ tree

```
├── README.md
├── bonus
├── docs
├── locals.tf
├── main.tf
├── modules
│   ├── networking
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   ├── security
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   └── webserver
│       ├── main.tf
│       ├── outputs.tf
│       └── variables.tf
├── outputs.tf
├── screenshots
│   ├── part1
│   ├── part2
│   ├── part3
│   ├── part4
│   ├── part5
│   └── part6
├── scripts
│   ├── apache-setup.sh
│   └── nginx-setup.sh
├── terraform.tfvars
└── variables.tf
```

15 directories, 17 files

✓ ASSIGNMENT2 [CODESPA...

✓ Assignment2

> .terraform

> aws

> docs

> modules

> screenshots

> scripts

📁 .gitignore

📄 .terraform.lock.hcl

📄 awscliv2.zip

🔑 LICENSE.txt

📄 locals.tf

📄 main.tf

📄 outputs.tf

📄 README.md

📄 terraform_1.14.0_linu..

📄 terraform.tfstate

📄 terraform.tfstate.bac...

📄 terraform.tfvars

📄 variables.tf

📄 README.md

📄 terraform.tfstate

📄 terraform.tfvars

GNU nano 7.2

.gitignore *

Terraform files

.terraform/

terraform.tfstate

terraform.tfstate.backup

*.tfvars

SSH keys

*.pem

*.key

id_rsa

id_ed25519

OS files

.DS_Store

GNU nano 7.2

README.md *

Assignment 2 - Advanced Terraform & Nginx Multi-Tier Architecture

This project deploys a high-availability multi-tier web infrastructure on AWS using Terraform

Components

- Nginx Reverse Proxy (HTTPS)
- 3 Apache Backend Servers
- Custom Security Groups
- Modular Terraform Code

More details will be added as the project progresses.

GNU nano 7.2

variables.tf *

```
variable "aws_region" {
  description = "AWS region where resources will be created"
  type        = string
  default     = "us-east-1"
}

variable "project_name" {
  description = "Name of the project"
  type        = string
  default     = "assignment2"
}

variable "vpc_cidr" {
  description = "CIDR block for VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "public_subnet_cidr" {
  description = "CIDR block for public subnet"
  type        = string
  default     = "10.0.1.0/24"
}

variable "private_subnet_cidr" {
  description = "CIDR block for private subnet"
  type        = string
  default     = "10.0.2.0/24"
}
```

```
GNU nano 7.2 terraform.tfvars *
key_pair_name = "assignment2-key"
```

```
GNU nano 7.2 locals.tf *
locals {
  common_tags = {
    Project      = var.project_name
    Environment  = "production"
    ManagedBy    = "Terraform"
  }
}
```

```
GNU nano 7.2 modules/networking/main.tf *
provider "aws" {
  region = var.aws_region
}

resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true

  tags = merge(
    local.common_tags,
    {
      Name = "${var.project_name}-vpc"
    }
  )
}

resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet_cidr
  map_public_ip_on_launch = true

  tags = merge(
    local.common_tags,
    {
      Name = "${var.project_name}-public-subnet"
    }
  )
}
```

```
GNU nano 7.2 modules/networking/variables.tf *
```

```
variable "aws_region" {  
    type = string  
}  
  
variable "project_name" {  
    type = string  
}  
  
variable "vpc_cidr" {  
    type = string  
}  
  
variable "public_subnet_cidr" {  
    type = string  
}  
█
```

```
GNU nano 7.2 modules/networking/outputs.tf *
```

```
output "vpc_id" {  
    value = aws_vpc.main.id  
}  
  
output "public_subnet_id" {  
    value = aws_subnet.public.id  
}  
█
```

root main.tf

```
GNU nano 7.2 main.tf *
terraform {
  required_version = ">= 1.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

module "networking" {
  source = "../modules/networking"

  aws_region      = var.aws_region
  project_name    = var.project_name
  vpc_cidr        = var.vpc_cidr
  public_subnet_cidr = var.public_subnet_cidr
}
```

```
GNU nano 7.2 outputs.tf *
output "vpc_id" {
  value = module.networking.vpc_id
}

output "public_subnet_id" {
  value = module.networking.public_subnet_id
}
```

security module

GNU nano 7.2

modules/security/main.tf *

```
resource "aws_security_group" "nginx_sg" {
  name           = "${var.project_name}-nginx-sg"
  description    = "Security group for Nginx reverse proxy"
  vpc_id         = var.vpc_id

  ingress {
    description = "Allow HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow HTTPS"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "Allow SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

GNU nano 7.2

modules/security/variables.tf *

```
variable "project_name" {
  type = string
}

variable "vpc_id" {
  type = string
}
```

GNU nano 7.2

modules/security/outputs.tf *

```
output "nginx_sg_id" {
  value = aws_security_group.nginx_sg.id
}

output "web_sg_id" {
  value = aws_security_group.web_sg.id
}
```

Proceed to connect security module

```
GNU nano 7.2 main.tf *
required_providers {
  aws = {
    source = "hashicorp/aws"
    version = "~> 5.0"
  }
}

provider "aws" {
  region = var.aws_region
}

module "networking" {
  source = "../modules/networking"

  aws_region      = var.aws_region
  project_name    = var.project_name
  vpc_cidr        = var.vpc_cidr
  public_subnet_cidr = var.public_subnet_cidr
}

module "security" {
  source = "../modules/security"

  project_name = var.project_name
  vpc_id       = module.networking.vpc_id
}

```

```
GNU nano 7.2 outputs.tf *
output "vpc_id" {
  value = module.networking.vpc_id
}

output "public_subnet_id" {
  value = module.networking.public_subnet_id
}

output "nginx_sg_id" {
  value = module.security.nginx_sg_id
}

output "web_sg_id" {
  value = module.security.web_sg_id
}

```

Ready for webserver module

```
GNU nano 7.2 scripts/apache-setup.sh *
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd

echo "<h1>Apache Server - $(hostname)</h1>" > /var/www/html/index.html
█
```

```
GNU nano 7.2 scripts/nginx-setup.sh *
#!/bin/bash
yum update -y
amazon-linux-extras enable nginx1
yum install -y nginx
systemctl start nginx
systemctl enable nginx

echo "<h1>Nginx Reverse Proxy</h1>" > /usr/share/nginx/html/index.html
█
```

```
resource "aws_instance" "nginx" {
  ami                = var.ami_id
  instance_type      = var.instance_type
  subnet_id          = var.public_subnet_id
  key_name            = var.key_pair_name
  vpc_security_group_ids = [var.nginx_sg_id]

  user_data = file("${path.module}/../../scripts/nginx-setup.sh")

  tags = merge(
    local.common_tags,
    {
      Name = "${var.project_name}-nginx"
    }
  )
}

resource "aws_instance" "web" {
  count      = 3
  ami        = var.ami_id
  instance_type = var.instance_type
  subnet_id  = var.public_subnet_id
  key_name    = var.key_pair_name
  vpc_security_group_ids = [var.web_sg_id]

  user_data = file("${path.module}/../../scripts/apache-setup.sh")

  tags = merge(
```

```
GNU nano 7.2 modules/webserver/variables.tf *
variable "project_name" {
    type = string
}

variable "ami_id" {
    type = string
}

variable "instance_type" {
    type = string
}

variable "public_subnet_id" {
    type = string
}

variable "key_pair_name" {
    type = string
}

variable "nginx_sg_id" {
    type = string
}

variable "web_sg_id" {
    type = string
}

```

```
GNU nano 7.2 modules/webserver/outputs.tf *
output "nginx_public_ip" {
    value = aws_instance.nginx.public_ip
}

output "web_private_ips" {
    value = aws_instance.web[*].private_ip
}

```

connect webserver module

GNU nano 7.2

main.tf *

```
project_name      = var.project_name
vpc_cidr          = var.vpc_cidr
public_subnet_cidr = var.public_subnet_cidr
}
```

```
module "security" {
  source = "./modules/security"

  project_name = var.project_name
  vpc_id       = module.networking.vpc_id
}
```

Nginx Server

```
module "nginx_server" {
  source           = "./modules/webserver"
  env_prefix       = var.env_prefix
  instance_name    = "nginx-proxy"
  instance_type    = var.instance_type
  availability_zone = var.availability_zone
  vpc_id           = module.networking.vpc_id
  subnet_id        = module.networking.subnet_id
  security_group_id = module.security.nginx_sg_id
  public_key       = var.public_key
  script_path      = "./scripts/nginx-setup.sh"
  instance_suffix  = "nginx"
  common_tags      = local.common_tags
}
```

```
availability_zone = var.availability_zone
vpc_id            = module.networking.vpc_id
subnet_id        = module.networking.subnet_id
security_group_id = module.security.nginx_sg_id
public_key        = var.public_key
script_path       = "./scripts/nginx-setup.sh"
instance_suffix   = "nginx"
common_tags       = local.common_tags
}
```

Backend Servers

```
module "backend_servers" {
  for_each = { for idx, server in local.backend_servers : server.name => server }

  source           = "./modules/webserver"
  env_prefix       = var.env_prefix
  instance_name    = each.value.name
  instance_type    = var.instance_type
  availability_zone = var.availability_zone
  vpc_id           = module.networking.vpc_id
  subnet_id       = module.networking.subnet_id
  security_group_id = module.security.backend_sg_id
  public_key       = var.public_key
  script_path      = each.value.script_path
  instance_suffix   = each.value.suffix
  common_tags      = local.common_tags
}
```

```
● @Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) $ terraform init
Initializing the backend...
Initializing modules...
- backend_servers in modules/webserver
- networking in modules/networking
- nginx_server in modules/webserver
- security in modules/security
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
○ @Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) $ █
```

```
GNU nano 7.2 terraform.tfvars
ami_id      = "ami-05524d6658fcf35b6"
key_pair_name = "assignment2-key"
```

```
● @Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) $ terraform validate
Success! The configuration is valid.
```

```
○ @Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) $ █
```

```
○ @Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) $ terraform plan █
```

```

    },
  ],
  + name                = "assignment2-web-sg"
  + name_prefix         = (known after apply)
  + owner_id            = (known after apply)
  + revoke_rules_on_delete = false
  + tags                = {
    + "Environment" = "production"
    + "ManagedBy"  = "Terraform"
    + "Name"        = "assignment2-web-sg"
    + "Project"     = "assignment2"
  }
  + tags_all            = {
    + "Environment" = "production"
    + "ManagedBy"  = "Terraform"
    + "Name"        = "assignment2-web-sg"
    + "Project"     = "assignment2"
  }
  + vpc_id              = (known after apply)
}

```

Plan: 23 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

+ nginx_sg_id      = (known after apply)
+ public_subnet_id = (known after apply)
+ vpc_id           = (known after apply)
+ web_sg_id        = (known after apply)

```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

@Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) \$ Microsoft Store

@Hafsa-Khalid021 → /workspaces/Assignment2/Assignment2 (main) \$ terraform plan -var-file="terraform.tfvars"

```

    + self              = false
    + to_port           = 80
  },
],
  + name                = "assignment2-web-sg"
  + name_prefix         = (known after apply)
  + owner_id            = (known after apply)
  + revoke_rules_on_delete = false
  + tags                = {
    + "Environment" = "production"
    + "ManagedBy"  = "Terraform"
    + "Name"        = "assignment2-web-sg"
    + "Project"     = "assignment2"
  }
  + tags_all            = {
    + "Environment" = "production"
    + "ManagedBy"  = "Terraform"
    + "Name"        = "assignment2-web-sg"
    + "Project"     = "assignment2"
  }
  + vpc_id              = "vpc-00a71a87014cbdfff"
}

```

Plan: 18 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```

~ nginx_sg_id      = "sg-018da6a49d931b459" -> (known after apply)
~ web_sg_id        = "sg-0ec132b56efcfff26" -> (known after apply)

```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

```
@Hafsa-Khalid021 →/workspaces/Assignment2/Assignment2 (main) $ terraform apply -auto-approve
module.security.aws_security_group.web_sg: Creation complete after 3s [id=sg-080954359f3e079dd]
module.networking.aws_subnet.public: Still creating... [00m10s elapsed]
module.networking.aws_subnet.public: Creation complete after 12s [id=subnet-09914fbaaa6d217d7]
module.networking.aws_route_table_association.public_assoc: Creating...
module.backend_servers["web-1"].aws_instance.web[0]: Creating...
module.backend_servers["web-1"].aws_instance.nginx: Creating...
module.nginx_server.aws_instance.web[0]: Creating...
module.nginx_server.aws_instance.nginx: Creating...
module.networking.aws_route_table_association.public_assoc: Creation complete after 0s [id=rtbassoc-06f72b28ed58be970]
module.backend_servers["web-1"].aws_instance.web[0]: Still creating... [00m10s elapsed]
module.backend_servers["web-1"].aws_instance.nginx: Still creating... [00m10s elapsed]
module.nginx_server.aws_instance.web[0]: Still creating... [00m10s elapsed]
module.nginx_server.aws_instance.nginx: Still creating... [00m10s elapsed]
module.nginx_server.aws_instance.nginx: Creation complete after 12s [id=i-022041e7748f26126]
module.backend_servers["web-1"].aws_instance.nginx: Creation complete after 12s [id=i-0f55e46a625e207fc]
module.nginx_server.aws_instance.web[0]: Creation complete after 12s [id=i-0f8021db16b566997]
module.backend_servers["web-1"].aws_instance.web[0]: Creation complete after 12s [id=i-0acfc2a16804c4c9b]
```

Apply complete! Resources: 11 added, 0 changed, 0 destroyed.

Outputs:

```
nginx_sg_id = "sg-06ec504bd26f1c173"
public_subnet_id = "subnet-09914fbaaa6d217d7"
vpc_id = "vpc-024730c2cf4d6e768"
web_sg_id = "sg-080954359f3e079dd"
```

```
@Hafsa-Khalid021 →/workspaces/Assignment2/Assignment2 (main) $ █
```

- @Hafsa-Khalid021 →/workspaces/Assignment2/Assignment2 (main) \$ terraform output


```
nginx_sg_id = "sg-06ec504bd26f1c173"
public_subnet_id = "subnet-09914fbaaa6d217d7"
vpc_id = "vpc-024730c2cf4d6e768"
web_sg_id = "sg-080954359f3e079dd"
```
- @Hafsa-Khalid021 →/workspaces/Assignment2/Assignment2 (main) \$ █

terraform.tfvars X locals.tf M variables.tf ../webserver M main.tf ../webserver M

Assignment2 > terraform.tfvars

```
1  block      = "10.0.0.0/16"
2  nr_block   = "10.0.10.0/24"
3  env        = "prod"
4  instance_type = "t3.micro"
5  key_path    = "/home/codespace/.ssh/id_ed25519.pub" # FIXED to absolute path
6  private_key = "/home/codespace/.ssh/id_ed25519"    # FIXED to absolute path
```

Assignment2 > outputs.tf

```
1  # Root outputs.tf
2
3  output "nginx_public_ip" {
4      description = "The public IP of the Nginx proxy server"
5      value        = module.nginx_server.public_ip
6  }
7
8  output "backend_private_ips" {
9      description = "A map of the backend server private IPs"
10     # This uses the for_each loop structure for the backend_servers module
11     value        = { for k, v in module.backend_servers : k => v.private_ip
12 }
```

Assignment2 > main.tf

```
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 6.27"
6     }
7   }
8   required_version = ">= 1.4.0"
9 }
10
11 provider "aws" {
12   region = var.region
13 }
14
15 # 1. NETWORKING MODULE (VPC, Subnet, IGW, Route Table)
16 module "networking" {
17   source           = "./modules/networking"
18   vpc_cidr_block   = var.vpc_cidr_block
19   subnet_cidr_block = var.subnet_cidr_block
20   availability_zone = var.availability_zone
21   env_prefix       = var.env_prefix
22   common_tags      = local.common_tags
23   region           = var.region # Ensure region is passed (to satisfy
24 }
```

ls.tf M × variables.tf .../webserver M main.tf .../webserver M

ment2 > locals.tf

```
data "http" "my_ip" {  
  url = "https://icanhazip.com"  
}
```

```
locals {  
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"  
  
  common_tags = {  
    Environment = var.env_prefix  
    Project      = "Assignment-2"  
    ManagedBy    = "Terraform"  
  }  
  
  backend_servers = [  
    {  
      name       = "web-1"  
      suffix     = "1"  
      script_path = "./scripts/apache-setup.sh"  
    },  
    {  
      name       = "web-2"  
      suffix     = "2"  
      script_path = "./scripts/apache-setup.sh"  
    },  
  ]  
}
```

... main.tf .../networking X outputs.tf .../networking main.tf .../security M variables.tf

HIC CHAINS... Assignment2 > modules > networking > main.tf

```
1 provider "aws" {
2   region = var.region
3 }
4
5 # Create VPC
6 resource "aws_vpc" "this" {
7   cidr_block = var.vpc_cidr_block
8   tags = {
9     Name      = "${var.env_prefix}-vpc"
10    Environment = var.env_prefix
11    ManagedBy  = "Terraform"
12  }
13 }
14
15 # Create Subnet
16 resource "aws_subnet" "this" {
17   vpc_id            = aws_vpc.this.id
18   cidr_block        = var.subnet_cidr_block
19   map_public_ip_on_launch = true
20   availability_zone  = var.availability_zone
21   tags = {
22     Name      = "${var.env_prefix}-subnet"
23     Environment = var.env_prefix
24     ManagedBy  = "Terraform"
25   }
26 }
27
28 # Create Internet Gateway
```

in.tf .../security M X

variables.tf .../security M

outputs.tf .../security M

\$ apac

ment2 > modules > security > main.tf

modules/security/main.tf - ONLY SECURITY GROUPS

1. Nginx Security Group

```
resource "aws_security_group" "nginx_sg" {  
  name          = "${var.env_prefix}-nginx-sg"  
  description   = "Allow SSH from my IP, HTTP/S from anywhere"  
  vpc_id       = var.vpc_id
```

Ingress: Port 22 (SSH) from your IP only

```
ingress {  
  from_port = 22  
  to_port   = 22  
  protocol  = "tcp"  
  cidr_blocks = [var.my_ip]  
}
```

Ingress: Port 80 (HTTP) from anywhere

```
ingress {  
  from_port = 80  
  to_port   = 80  
  protocol  = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}
```

Egress: All traffic

```
egress {  
  from_port = 0  
  to_port   = 0
```

In 75 Col 2 Spaces: 2

```

resource "aws_security_group" "nginx_sg" {
  # Egress: All traffic
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = merge(var.common_tags, { Name = "${var.env_prefix}-nginx-sg" })
}

```

2. Backend Security Group

```

resource "aws_security_group" "backend_sg" {
  name           = "${var.env_prefix}-backend-sg"
  description    = "Allow SSH from my IP, HTTP from Nginx SG only"
  vpc_id         = var.vpc_id

  # Ingress: Port 22 (SSH) from your IP only (for you to debug)
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
  }

  # --- CRITICAL FIX ADDED HERE ---
  # Ingress: Port 22 (SSH) from NGINX Security Group ONLY (for jump host)

```

```

    to_port      = 22
    protocol     = "tcp"
    security_groups = [aws_security_group.nginx_sg.id] # Source is Nginx
  }
# --- END CRITICAL FIX ---

# Ingress: Port 80 (HTTP) from Nginx security group ONLY
ingress {
  from_port      = 80
  to_port        = 80
  protocol       = "tcp"
  security_groups = [aws_security_group.nginx_sg.id] # Source is Nginx
}

# Egress: All traffic
egress {
  from_port      = 0
  to_port        = 0
  protocol       = "-1"
  cidr_blocks    = ["0.0.0.0/0"]
}
tags = merge(var.common_tags, { Name = "${var.env_prefix}-backend-sg"

```

```

@Hafsa-Khalid021 →/workspaces/Assignment2/Assignment2 (main) $ terraform apply -auto-approve
module.security.aws_security_group.web_sg: Creation complete after 3s [id=sg-080954359f3e079dd]
module.networking.aws_subnet.public: Still creating... [00m10s elapsed]
module.networking.aws_subnet.public: Creation complete after 12s [id=subnet-09914fbaaa6d217d7]
module.networking.aws_route_table_association.public_assoc: Creating...
module.backend_servers["web-1"].aws_instance.web[0]: Creating...
module.backend_servers["web-1"].aws_instance.nginx: Creating...
module.nginx_server.aws_instance.web[0]: Creating...
module.nginx_server.aws_instance.nginx: Creating...

```

Outputs:

```

backend_private_ips = {
  "web-1" = "10.0.10.69"
  "web-2" = "10.0.10.20"
  "web-3" = "10.0.10.164"
}
nginx_public_ip = "51.112.186.38"

```

Welcome to Nginx on ip-10-0-10-122.me-central-1.compute.internal

```
@Hafsa-Khalid021 → /workspaces/Assignment2 (main) $ ssh -i /home/
```

```
    _/m/'
```

```
Last login: Tue Dec 30 14:27:05 2025 from 20.192.21.54
```

```
[ec2-user@ip-10-0-10-122 ~]$ sudo nginx -t
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
```

```
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
[ec2-user@ip-10-0-10-122 ~]$ sudo service nginx restart
```

```
Redirecting to /bin/systemctl restart nginx.service
```

```
[ec2-user@ip-10-0-10-122 ~]$ exit
```

```

• @Hafsa-Khalid021 →/workspaces/Assignment2 (main) $ cd /workspaces/Assignment2
• @Hafsa-Khalid021 →/workspaces/Assignment2 (main) $ git rm --cached Assignment2/aws/dist/awscli/examples/gre
  engrass/create-connector-definition-version.rst
  git commit -m "Remove file containing secret"
  rm 'Assignment2/aws/dist/awscli/examples/greengrass/create-connector-definition-version.rst'
  [main 8e9976d] Remove file containing secret
    1 file changed, 16 deletions(-)
    delete mode 100644 Assignment2/aws/dist/awscli/examples/greengrass/create-connector-definition-version.rst
• @Hafsa-Khalid021 →/workspaces/Assignment2 (main) $ git rm --cached Assignment2/awscliv2.zip
  git commit -m "Remove large zip file to meet GitHub limits"
  rm 'Assignment2/awscliv2.zip'
  [main 7001006] Remove large zip file to meet GitHub limits
    1 file changed, 0 insertions(+), 0 deletions(-)
    delete mode 100644 Assignment2/awscliv2.zip
• @Hafsa-Khalid021 →/workspaces/Assignment2 (main) $ git lfs install
  git lfs track "Assignment2/awscliv2.zip"
  git add .gitattributes
  git add Assignment2/awscliv2.zip
  git commit -m "Track large AWS CLI zip using Git LFS"
  Updated Git hooks.
  Git LFS initialized.
  Tracking "Assignment2/awscliv2.zip"
  [main a079366] Track large AWS CLI zip using Git LFS
    2 files changed, 4 insertions(+)

```

← → ↻ ⚠ Not secure 51.112.186.38

Welcome to Apache on ip-10.0.10.249

PROBLEMS OUTPUT TERMINAL ...

ssh - Assignment2 ⚠ + ▾ 🗑 ... | 📄 ✕

GNU nano 8.3

/etc/nginx/nginx.conf

Modified

```

}
upstream backend_group {
    server 10.0.10.249:80;    # Primary
    server 10.0.10.32:80;    # Primary
    server 10.0.10.117:80 backup; # Backup ONLY
}

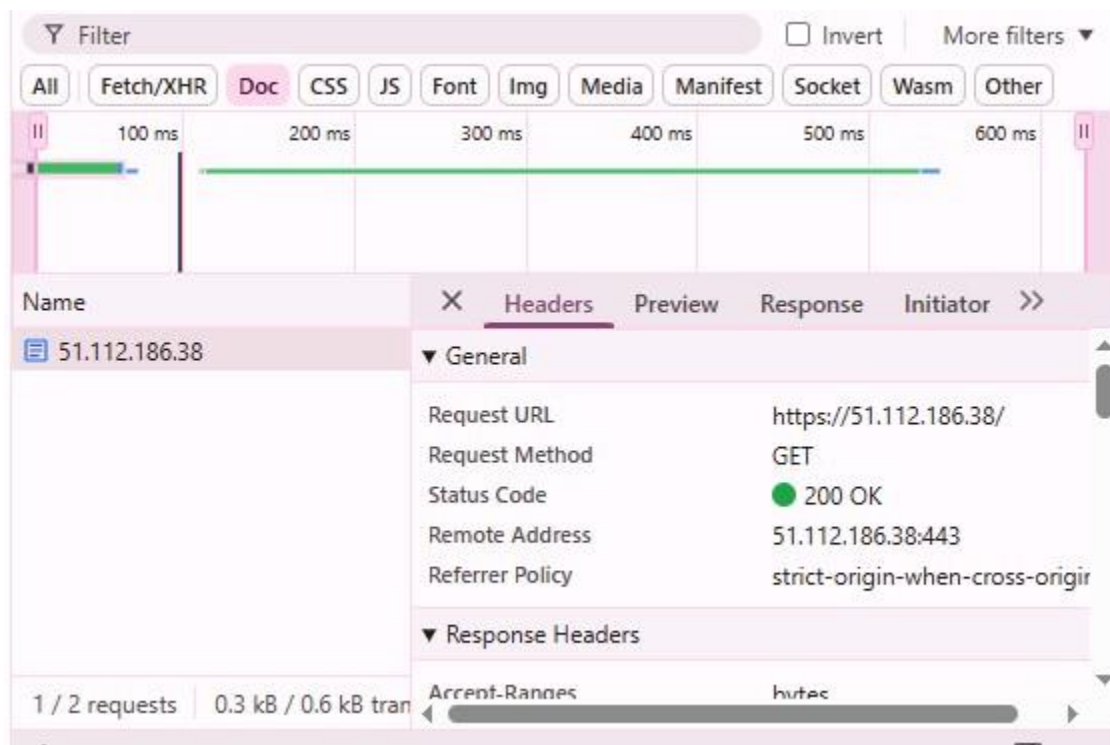
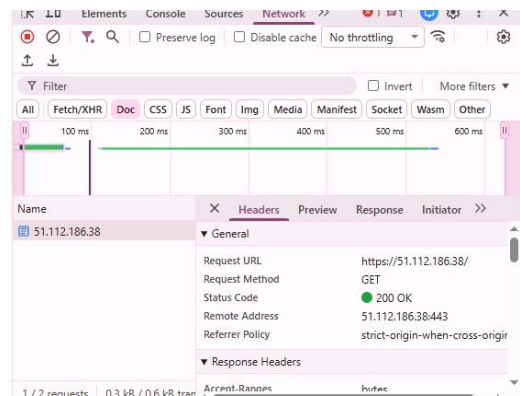
```

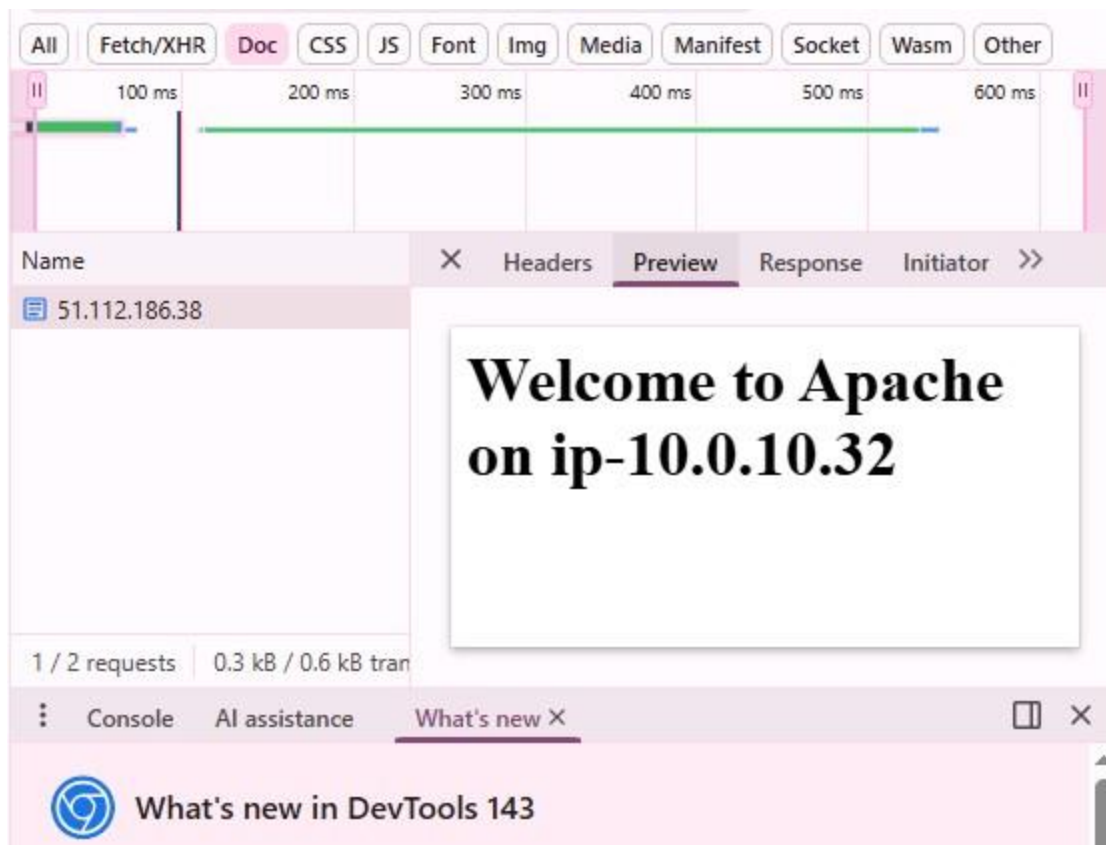
Instances (6) [Info](#) Refresh Connect Instance state Actions Launch instances

Find Instance by attribute or tag (case-sensitive) All states < 1 > Settings

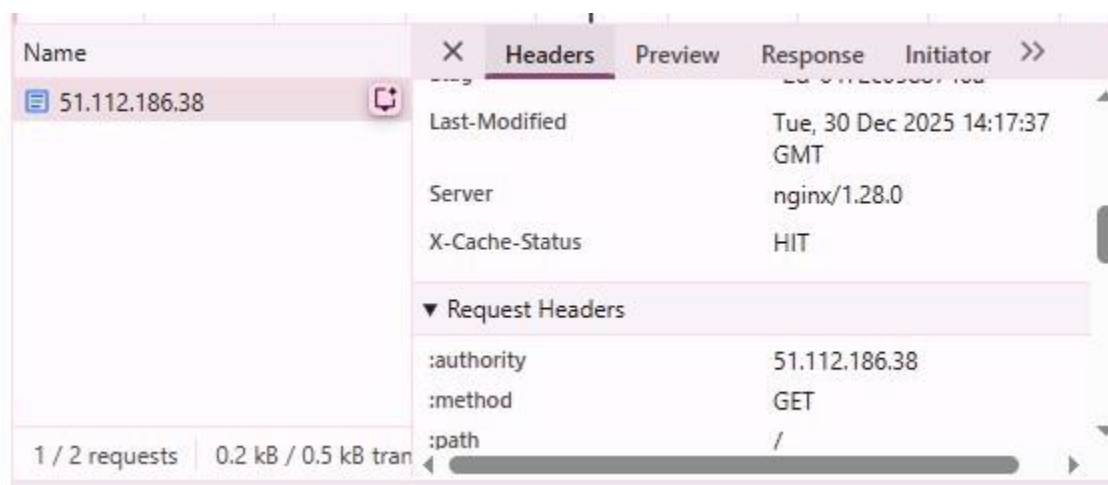
☐	Name ↗	Instance ID	Instance state ↗	Instance type ↗	Status check	Alarm status	Availability
☐	nginx-server	i-0a59bc2a26c91fd1d	Running 🔍 🔍	t3.micro	3/3 checks passed	View alarms +	me-central
☐	web-server	i-00ca19afdc23dcc9b	Running 🔍 🔍	t3.micro	3/3 checks passed	View alarms +	me-central
☐	prod-nginx-pr...	i-010989d728ba10231	Running 🔍 🔍	t3.micro	3/3 checks passed	View alarms +	me-central
☐	prod-web-1	i-0c46c946c9100f2f0	Running 🔍 🔍	t3.micro	3/3 checks passed	View alarms +	me-central

Welcome to Apache on ip-10.0.10.32

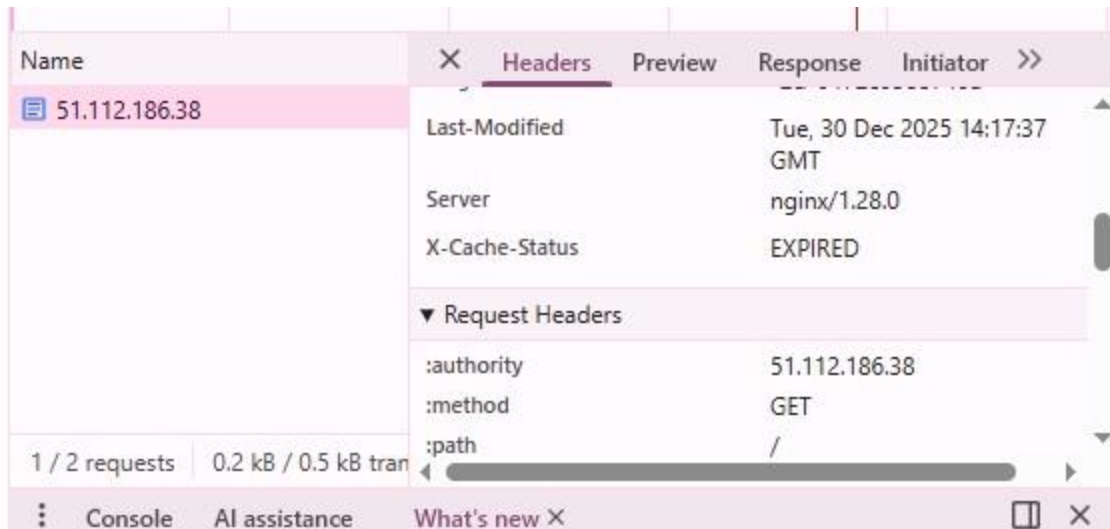




Cache HIT



Cache Miss



HTTP Permanently moved

```
@Hafsa-Khalid021 → /workspaces/Assignment2 (main) $ curl -I http://51.112.186.38
HTTP/1.1 301 Moved Permanently
Server: nginx/1.28.0
Date: Tue, 30 Dec 2025 18:03:04 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://51.112.186.38/
```

<https://github.com/Hafsa-Khalid021/Assignment2>